

Object & Class Diagramming

When creating a system, we want to know what THINGS (*Objects*) we're going to have in our system. "Things" are literally just nouns AKA objects, and you have to decide from the **Fact Finding** that we ran, (Use Case Diagrams, Activity Diagrams, Interviews) What things we're going to have.

- **Tangible Things** (*Book*)
 - **Employee Role** (*Reception, Manager*)
 - **Organization/Places** (*Health Department, Factory, Lab*)
 - Then on top of that, there's **EVENTS** (*A Checkout, A Fine on overdue book*)
 - Then there's **Interactions** (*Two THINGS, i.e. Order & Book make checkout*)
-

There are a few key things about **OBJECTS** that *we need to know for right now*; objects have attributes, i.e. things that describe it, the *amount of pages, the author name, the genre.* They also *can (DON'T have to be) be related to other objects.*

- And **those relations**, are described with **NUMBERS**; called cardinality.
-

To Briefly understand this, let's say we have the **object "Book"** and the **object "Order"**. Let's say *an order can have more than one book*, but must at least have one book. A book can either never be ordered, or ordered as many times as you want.

> "ORDER can have more than one book, but must at least have one book."

- Cardinality is described as **min to max, 1 minimum, unlimited maximum.**
 - Which is described as **1..*** (** means unlimited*) FOR THE ORDER OBJECT
 - But we have to describe BOTH objects

> "A **book** can either never be ordered, or ordered as many times as you want.

- Again, *minimum to maximum*; so NEVER be ordered is 0, to as many as you want, so *
 - We **describe it** as **0..***
-

Now, We want to take all of this information and **ADD it into a diagram.**

- This is called a **“Class Diagram”**

Let's say, we're in a library; it has books, customers can *CHECKOUT*, each checkout is made up of a "**Checkout Entry**" which describes the *QUANTITY* of a book you got, for each part of your entire checkout; each entry has one book, which is described via Book Name, Book ID (which is called an ISBN)

- For **RELATED** objects, *we draw lines between them.*
- Then we grab the **CARDINALITY** (i.e. that $0..*$ or $1..*$)

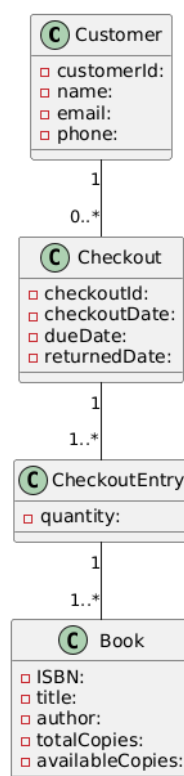
So our objects are **BOOK**, **CUSTOMER**, **CHECKOUT**, **CHECKOUT-ENTRY**

Let's establish our **CARDINALITIES**,

- Customer & Checkout, a customer can have 0 checkouts, or as many as they want. But a checkout can only be from one customer,
 - MEANING, **Customer 1;0..* Checkout**
 - (*Is customer related to anything else? Checkout-entry? No. Book? No.*)
- Checkout & Checkout-Entry, there can only be one checkout; but every checkout has MANY *Checkout-Entries*
 - MEANING, **Checkout 1;1..* Checkout-Entry**
 - (*Is Checkout, related to anything else Customer? Done. Book? No*)
- Checkout-Entry & Book, Checkout-Entry needs *One Book to exist* (multiple of the same is stored in quantity.) Book, can be in as many Checkout-Entries as it wants, or none (i.e. many customers checkout, or none, or just one)
 - MEANING, **Checkout-Entry Book**
 - (*Is Checkout-Entry related to anything else? Customer? No. Checkout? Done.*)

With this we've established every **RELATION** and **CARDINALITY**.

- And can make the diagram below (*I added some extra detail*)



The hard part of diagramming isn't drawing it, it's pretty obvious that all we need to do is draw a line if two objects are related, and add the **CARDINALITY**.

- The hard part is dissecting the information to see where those **relations and cardinalities lie**. (Which will get easier with time, try practicing with GPT)
-

Now, that diagram from before is actually quite HIGH LEVEL. After we get further into development, we want to add more detail.

- This is a two step process;
- **First**, you have to understand something called the "Three Tier Architecture".

Three Tier Architecture: What the three tiered architecture does is try to make your objects match a real system.

A real system will **DIVIDE**, different parts of a system

- What the **USER** sees (1. "View")
 - A Shopping Cart
- What the **system** handles (2. "Problem Domain")
 - The actual Checkout-Entry
- And the **actual Data** (3. "Data Layer")
 - The entire Library Database, The Customer

This architecture *mimics real system design* via something called a "**STEREOTYPE**", Each tier of the Three Tier Architecture has unique stereotypes.

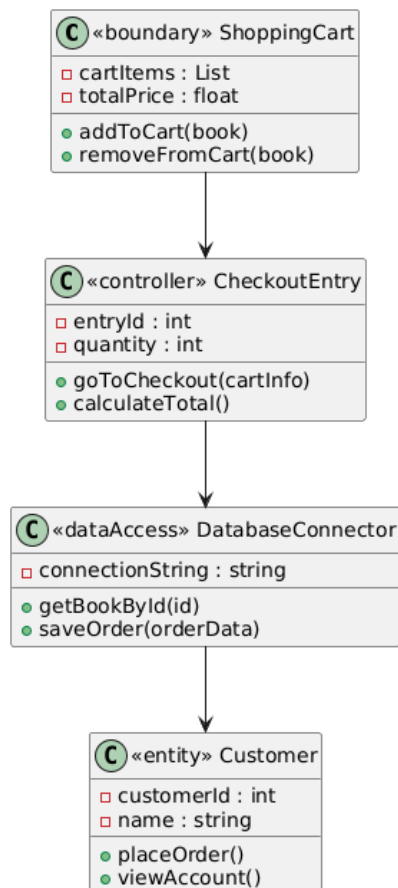
- The "View" tier, has a Stereotype called **<<BOUNDARY>>**, this is because it sits between a customer, and the actual system. Like a shopping cart.
 - The "Problem Domain" tier, has a Stereotype called **<<CONTROLLER>>**, controller because it **CONTROLS** the actual logic of the system. The **BUSINESS** rules of the system.
 - The "Data" tier, has **TWO** stereotypes, **<<entity>>**, is a person; like a customer or librarian. And **<<dataAccess>>** is a stereotype for that entity to access data (in between Problem Domain & Entity).
 - Through an object that has the **<<dataAccess>>** stereotype the **<<entity>>** object can actually access the entire library database to search for books.
-

The SECOND thing, is to add METHODS; *an object needs to be able to DO things*. A customer needs to be able to do the action "Add to Shopping cart", or the action "Proceed to checkout"

- This is rather simple to add, *only thing is METHODS (just think of as actions)*, sometimes need information to do an action. If you want to go to checkout, I need the Shopping Cart information, so you FEED that into the Method.
 - Which looks like `goToCheckout(shoppingCartInformation)`

Neat little thing, we can actually REMOVE the cardinalities now, all we need is the relation lines.

So this is called a **FIRST CUT Domain Class Diagram**, *this is a diagram that developers can MORE READILY USE*, to actually develop the system.



- Some of the attributes from before now have "string" or "int" or "float", that just means what type of value is that? The `totalPrice` is Float (Decimal Number)
-

The next part of this lab will be the last, it will teach **SEQUENCE diagrams**; which is effectively **MORE IN DEPTH activity diagrams**.